Cloud engineering

M2 MOSEF

Juan, Victoria, Tasnim, Jorel, Mélina, Amel

01

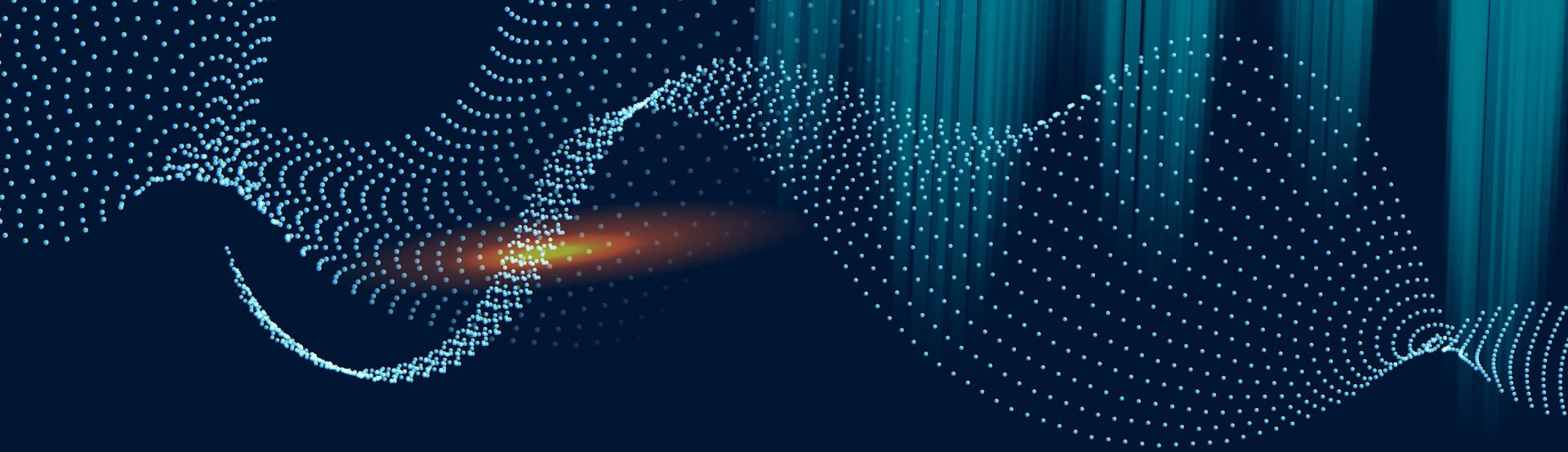
KAFKA & OPENSEARCH

Flux de données & Visualisations

02

GCP

Ingestion de donnée sur BigQuery & Requêtage avec SQL



01 KAFKA & OPENSEARCH

Comment faire des visualisations à partir de données en flux ?

PRODUCTEUR
Envoi de messages en format JSON dans le topic kafka.

CONSOMMATEUR
Lis les messages disponibles dans le topic et les envoie vers OpenSearch, qui les stocke dans un index.

INDEX PATTERN
On choisit l'index (la table) rattaché aux visualisations sur OpenSearch.

CREATION DU SCHEMA
On renseigne le type de chaque colonne dans OpenSearch pour faciliter la lecture des tables avec un index pattern.

VISUALISATIONS
Création d'un dashboard interactif basé sur des données en flux.

Le pipeline complet d'une architecture de data streaming.

SCRIPT PYTHON - PRODUCTEUR

L'adresse du **broker Kafka** . C'est le point d'entrée vers Kafka.

Kafka ne comprend que des **bytes**, pas du JSON directement. On sérialise les messages.

On envoie le message vers le **topic Kafka** "victoria".

On affiche l'**offset** , donc l'identifiant du message envoyé.

```python
from kafka import KafkaProducer
import json
from datetime import datetime, timezone

producer = KafkaProducer(
    bootstrap_servers=['vps-fb43df0a.vps.ovh.net:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

message = {
    "location": "10.76, -14.76",
    "typeproduit": "legume",
    "prix": 200.75,
    "agent_timestamp": datetime.now(timezone.utc).strftime('%Y-%m-%dT%H:%M:%SZ')
}



future = producer.send('victoria', value=message)
record_metadata = future.get(timeout=10)
print("Envoyé - Offset:", record_metadata.offset)
producer.flush()
producer.close()
```

Le producteur envoie des données (ici, un message) en format JSON vers un topic Kafka.

SCRIPT PYTHON - CONSOMMATEUR

Le consommateur désérialise les messages pour **récupérer un format JSON** à partir de bytes.

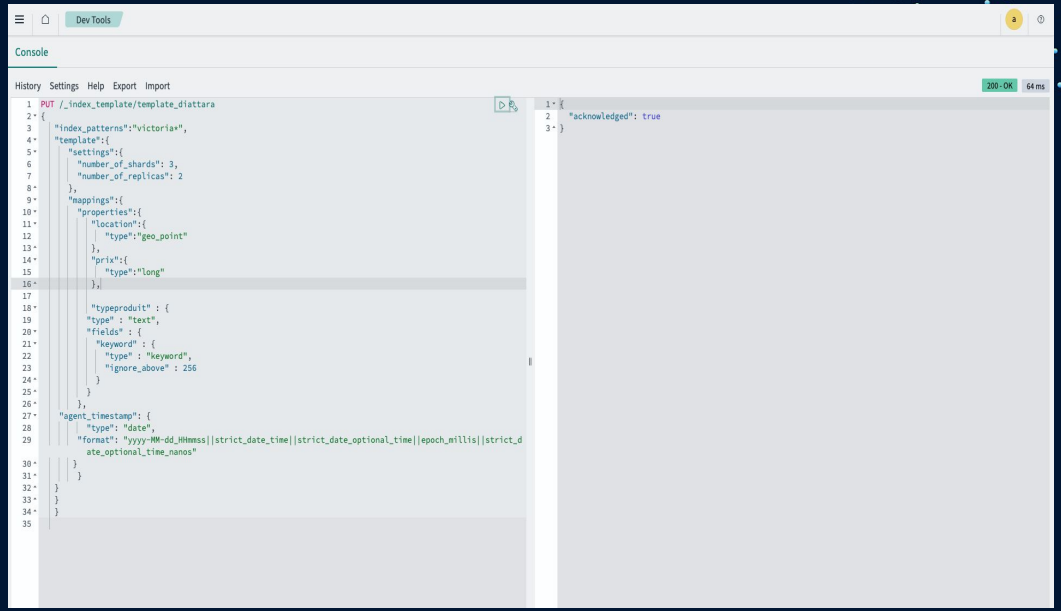
Si le consommateur n'a jamais lu ce topic, il commence depuis le **tout premier message** disponible dans la partition.

L'URL pointe vers notre index dans OpenSearch avec le **endpoint /_doc** pour insérer un document.

```python
from kafka import KafkaConsumer
import requests, json, urllib3

urllib3.disable_warnings()

consumer = KafkaConsumer(
    'victoria',
    bootstrap_servers=['vps-fb43df0a.vps.ovh.net:9092'],
    #group_id='groupe_1',
    value_deserializer=lambda v: json.loads(v.decode('utf-8')),
    auto_offset_reset='earliest'
)

for msg in consumer:
    document = msg.value
    response = requests.post(
        'https://vps-897125f3.vps.ovh.net:9200/victoria/_doc',
        json=document,
        headers={'Content-Type': 'application/json'},
        auth=('admin', 'ChangeMoi123!'),
        verify=False
    )
    print(f" Indexé (offset {msg.offset}) - status {response.status_code}")
```

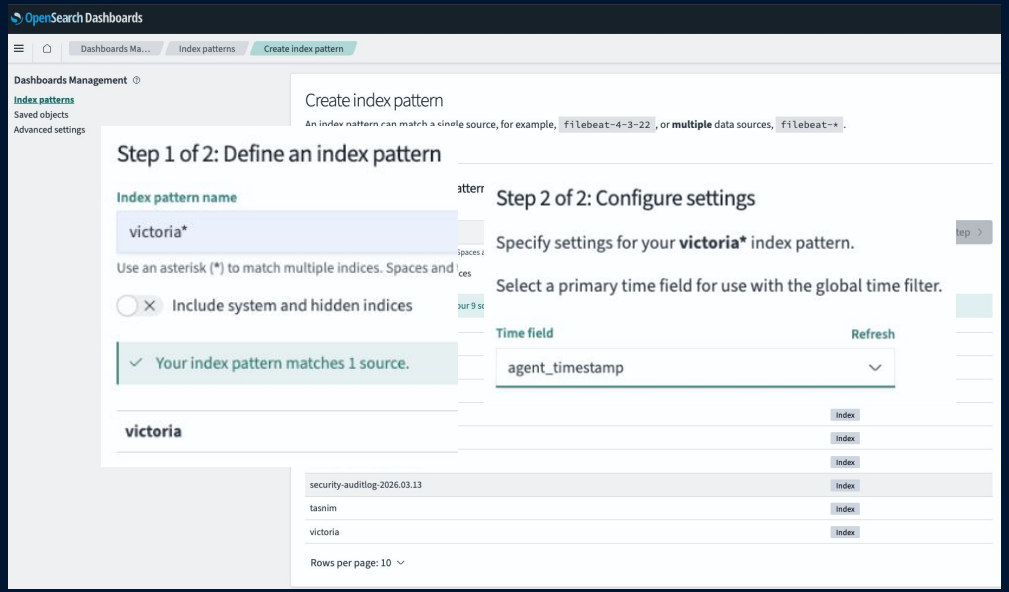
Le consommateur lit les données stockées dans le topic Kafka et les envoie sur OpenSearch qui les stocke dans un index.

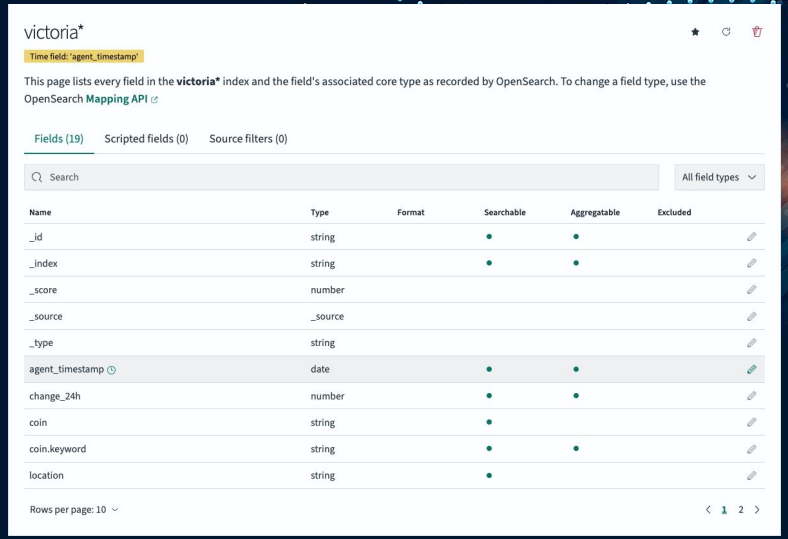
CREATION DU SCHEMA



Définition du mapping OpenSearch et visualisation des champs typés (geo_point, long, text, date), permettant d'optimiser le stockage, la recherche et l'analyse des données.

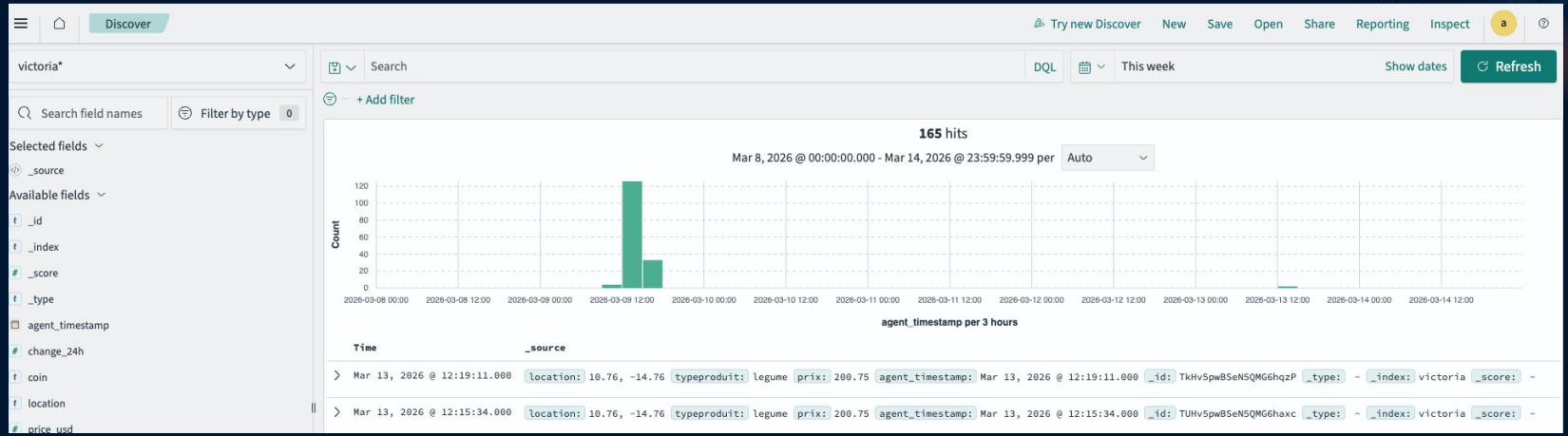
SELECTION DE L'INDEX





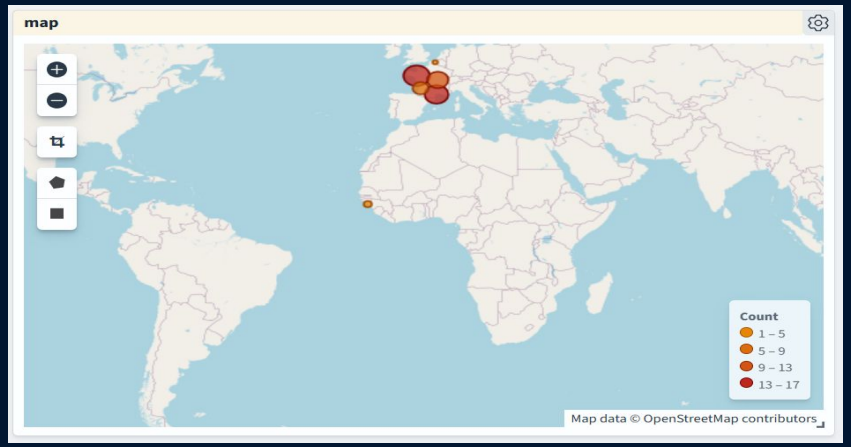
Création et configuration d'un index pattern (nom et champ temporel) permettant de structurer les données et de les rendre exploitables pour l'analyse.

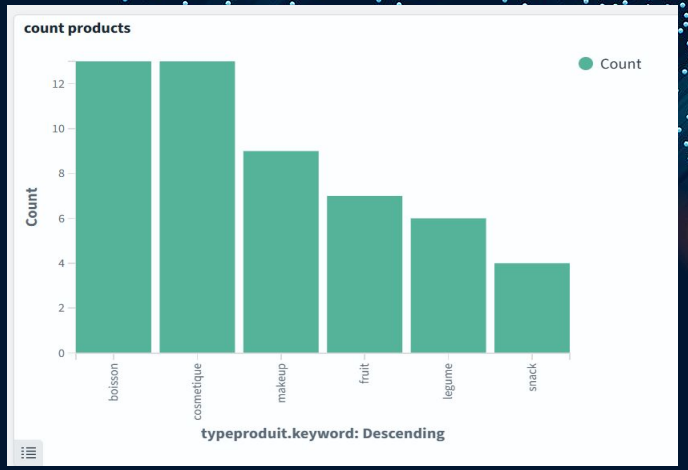
TABLEAU DE BORD

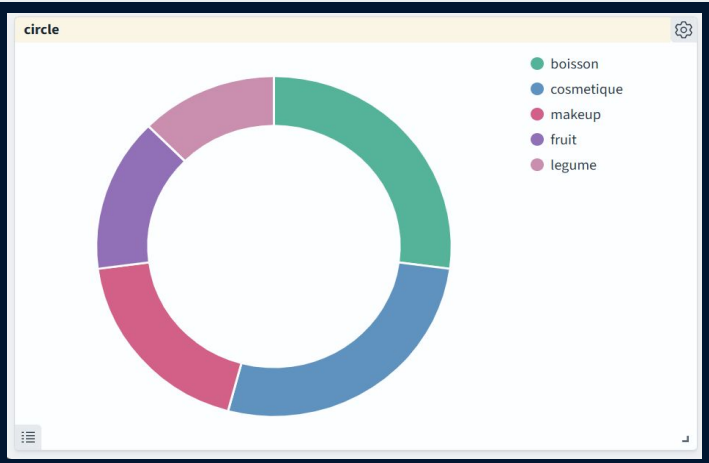


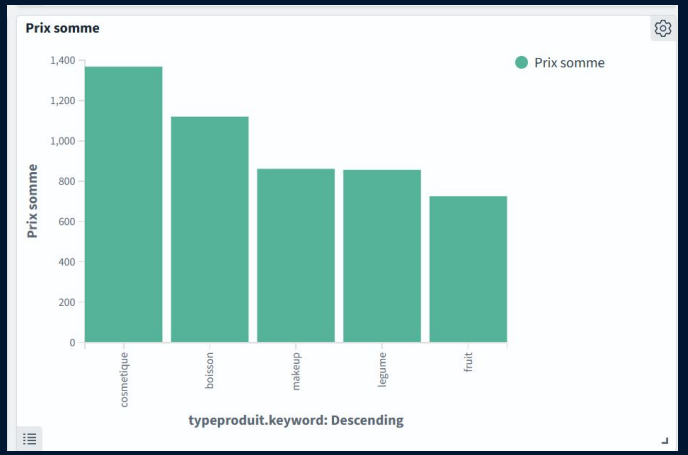
Exploration des données dans OpenSearch (Discover) permettant de visualiser les événements, filtrer les données et analyser leur évolution dans le temps.

TABLEAU DE BORD









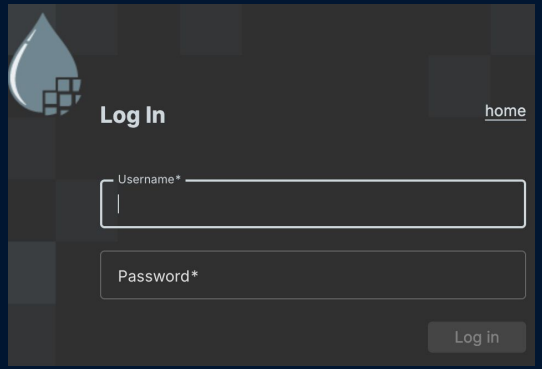
NIFI – Automatisation du pipeline

Mise en place d'un flux automatisé Kafka → OpenSearch, sans code.

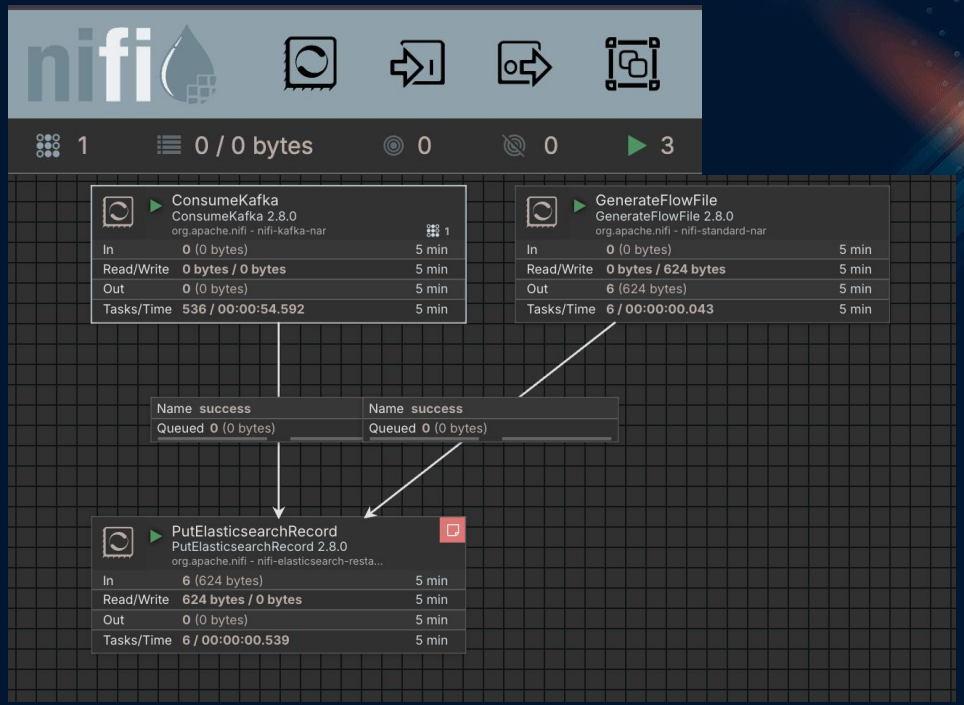
Démarrage de NiFi

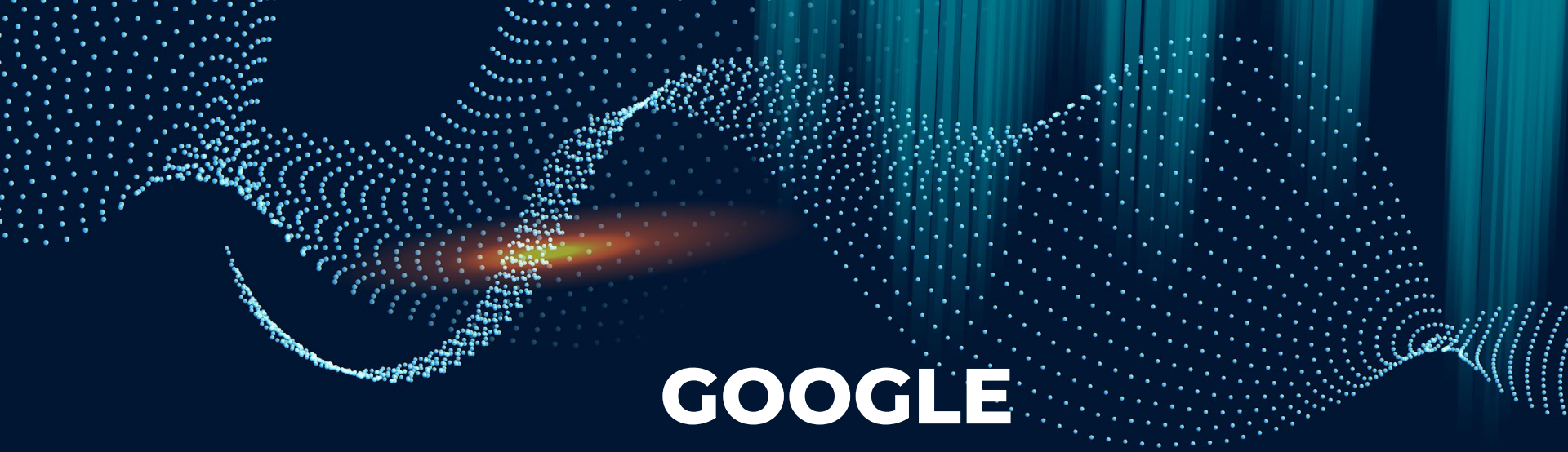
```
victoria.v@MacBook-Pro-de-Victoria-Vidal-Eudier ~ % cd Downloads/
victoria.v@MacBook-Pro-de-Victoria-Vidal-Eudier Downloads % cd nifi-2.8.0
victoria.v@MacBook-Pro-de-Victoria-Vidal-Eudier nifi-2.8.0 % cd bin
victoria.v@MacBook-Pro-de-Victoria-Vidal-Eudier bin % ./nifi.sh start

JAVA_HOME=/Users/victoria.v/.sdkman/candidates/java/current
NIFI_HOME=/Users/victoria.v/Downloads/nifi-2.8.0
```

Connexion



Construction du flux





02 GOOGLE CLOUD PLATFORM

Comment stocker des données dans le cloud et les analyser avec SQL ?

Envoi de données vers BigQuery

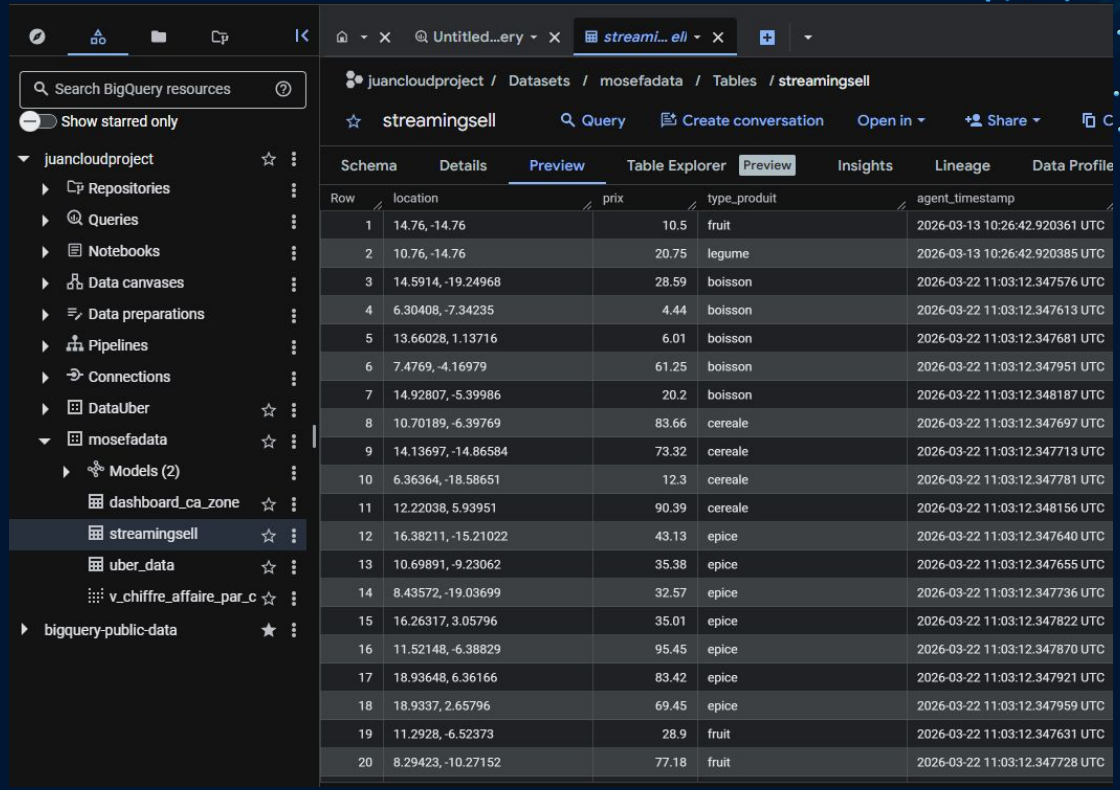
```python
from google.cloud import bigquery
from google.oauth2 import service_account
from datetime import datetime, UTC
import json
import tempfile
import os
import random

PROJECT_ID = "juancloudproject"
TABLE_ID = "mosefadata.streamingsell"
KEY_PATH = "/content/juancloudproject-51f364a85501.json"

credentials = service_account.Credentials.from_service_account_file(KEY_PATH)
client = bigquery.Client(credentials=credentials, project=PROJECT_ID)

random.seed(42)

type_produits = [
    "fruit",
    "legume",
    "cereale",
    "boisson",
    "epice",
    "poisson",
    "viande",
    "laitier",
    "snack",
    "huile"
]

rows_to_insert = []

for _ in range(60):
    latitude = round(random.uniform(5.0, 20.0), 5)
    longitude = round(random.uniform(-20.0, 10.0), 5)
    prix = round(random.uniform(1.5, 100.0), 2)
    type_produit = random.choice(type_produits)

    rows_to_insert.append({
        "location": f"{latitude}, {longitude}",
        "prix": prix,
        "type_produit": type_produit,
        "agent_timestamp": datetime.now(UTC).isoformat()
    })
```

```python
job_config = bigquery.LoadJobConfig(
    source_format=bigquery.SourceFormat.NEWLINE_DELIMITED_JSON,
    write_disposition=bigquery.WriteDisposition.WRITE_APPEND
)

tmp_file = None

try:
    with tempfile.NamedTemporaryFile(mode="w", suffix=".json", delete=False, encoding="utf-8") as f:
        tmp_file = f.name
        for row in rows_to_insert:
            f.write(json.dumps(row) + "\n")

    with open(tmp_file, "rb") as source_file:
        job = client.load_table_from_file(
            source_file,
            TABLE_ID,
            job_config=job_config
        )

    job.result()
    print("Chargement réussi dans BigQuery")
    print(f"{len(rows_to_insert)} lignes insérées")

except Exception as e:
    print("Erreur :", repr(e))

finally:
    if tmp_file and os.path.exists(tmp_file):
        os.remove(tmp_file)
```

```
Chargement réussi dans BigQuery
60 lignes insérées
```

Ce script charge dans la table mosefadata.streamingsell de BigQuery au format JSON, avec confirmation du nombre de lignes insérées.

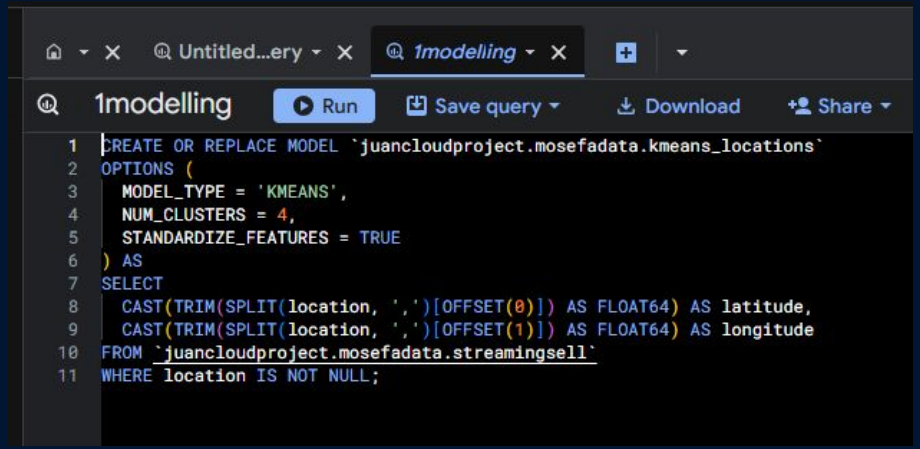
Vérification du chargement des données dans BigQuery

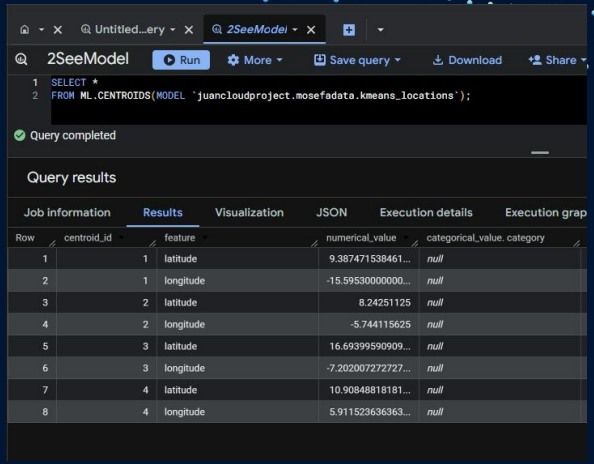


Row	location	prix	type_produit	agent_timestamp
1	14.76, -14.76	10.5	fruit	2026-03-13 10:26:42.920361 UTC
2	10.76, -14.76	20.75	legume	2026-03-13 10:26:42.920385 UTC
3	14.5914, -19.24968	28.59	boisson	2026-03-22 11:03:12.347576 UTC
4	6.30408, -7.34235	4.44	boisson	2026-03-22 11:03:12.347613 UTC
5	13.66028, 1.13716	6.01	boisson	2026-03-22 11:03:12.347681 UTC
6	7.4769, -4.16979	61.25	boisson	2026-03-22 11:03:12.347951 UTC
7	14.92807, -5.39986	20.2	boisson	2026-03-22 11:03:12.348187 UTC
8	10.70189, -6.39769	83.66	cereale	2026-03-22 11:03:12.347697 UTC
9	14.13697, -14.86584	73.32	cereale	2026-03-22 11:03:12.347713 UTC
10	6.36364, -18.58651	12.3	cereale	2026-03-22 11:03:12.347781 UTC
11	12.22038, 5.93951	90.39	cereale	2026-03-22 11:03:12.348156 UTC
12	16.38211, -15.21022	43.13	epice	2026-03-22 11:03:12.347640 UTC
13	10.69891, -9.23062	35.38	epice	2026-03-22 11:03:12.347655 UTC
14	8.43572, -19.03699	32.57	epice	2026-03-22 11:03:12.347736 UTC
15	16.26317, 3.05796	35.01	epice	2026-03-22 11:03:12.347822 UTC
16	11.52148, -6.38829	95.45	epice	2026-03-22 11:03:12.347870 UTC
17	18.93648, 6.36166	83.42	epice	2026-03-22 11:03:12.347921 UTC
18	18.9337, 2.65796	69.45	epice	2026-03-22 11:03:12.347959 UTC
19	11.2928, -6.52373	28.9	fruit	2026-03-22 11:03:12.347631 UTC
20	8.29423, -10.27152	77.18	fruit	2026-03-22 11:03:12.347728 UTC

Comme on peut le voir dans l'onglet Preview de la table, les nouvelles lignes insérées sont bien visibles, ce qui permet de vérifier que la table a été mise à jour avec succès.

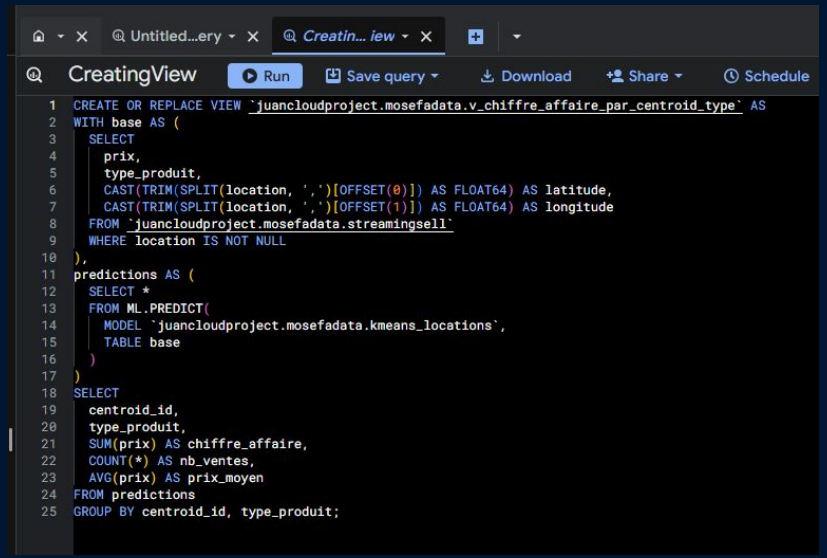
Création et visualisation du modèle K-means dans BigQuery ML





La première requête permet de créer un modèle de clustering K-means à partir des coordonnées extraites de la table streamingsell, tandis que la seconde affiche les centroïdes calculés pour chaque cluster, afin de visualiser les différentes zones géographiques identifiées par le modèle.

Création d’une vue analytique pour le chiffre d’affaires par zone et type de produit



Cette requête crée une vue BigQuery qui associe chaque vente à un centroïde du modèle K-means, puis calcule pour chaque zone et type de produit le chiffre d’affaires total, le nombre de ventes et le prix moyen.

Consultation des résultats de la vue analytique

Untitled...ery | SeeingView

SeeingView | Run | Save query | Download | Share | Schedule | Open

```
1  SELECT *
2  FROM `juancloudproject.mosefadata.v_chiffre_affaire_par_centroid_type`
3  ORDER BY centroid_id, chiffre_affaire DESC;
```

Query completed

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	centroid_id	type_produit	chiffre_affaire	nb_ventes	prix_moyen
1	1	snack	241.9899999999...	3	80.66333333333...
2	1	legume	105.93	3	35.31
3	1	huile	93.65	1	93.65
4	1	viande	89.65	2	44.825
5	1	epice	32.57	1	32.57
6	1	boisson	28.59	1	28.59
7	1	cereale	12.3	1	12.3
8	1	fruit	7.81	1	7.81
9	2	legume	168.0	2	84.0
10	2	snack	133.54	4	33.385
11	2	epice	130.83	2	65.415
12	2	fruit	106.0800000000...	2	53.04000000000...
13	2	viande	89.94	1	89.94
14	2	laitier	88.64	1	88.64
15	2	cereale	83.66	1	83.66
16	2	boisson	65.69	2	32.845

Cette requête permet d'afficher le contenu de la vue v_chiffre_affaire_par_centroid_type et de classer les résultats par centroïde puis par chiffre d'affaires décroissant, afin d'identifier rapidement les types de produits les plus performants dans chaque zone.

Merci pour votre attention.